PA1 REPORT

jade-lynx-c0ee

The completion of the assignment:

All

How to compile and execute your program:

我設計了八個函數:
函數1:輸入價格
函數2:資料設置(d/p)、(y/n)
函數3:計算價格
函數4:判斷是否有足夠的券付
函數5:付錢的方法(最主要的fuction)
函數6:顯示剩餘的券
函數7:顯示使用的券
函數8:計算券還有多少錢

```
#include <iostream>
#include<string>
using namespace std;

void price_set(int []);                    //    function begin
void data_set(string []);                  //     set price
void price_count(int [],string []);        //set data
bool price_judge(int [],int []);           //count price
void price_payment(int [],int []);         //judge price
void voucher_show_remain(int []);          //the method of payment
void voucher_show_use(int []);             //show vouchers
int voucher_count(int[]);                  //    count remain vouchers
```

主程式

```
int main(void)
{
    int price[1];                   // 價格
    int voucher[5]={5,5,5,5,5}; //1$,5$,10$,50$,100$張數
    string  data[2];                    //(d/n)、(y/n)

    cout<<"*** Welcome to Quintuple Stimulus Vouchers System ***"<<endl;
    while(price[0]!=-1&&voucher_count(voucher)!=0)
    {
        price_set(price);
        if(price[0]==-1)            //價格為-1結束迴圈
        {
            cout<<"Bye~"<<endl;
            break;
        }
        data_set(data);             //輸入資料
        price_count(price,data);    //價格計算
        if(price_judge(price,voucher)==false)     //判斷是否有足夠的錢的付，為否即回到一開始
        {
            cout<<"You don't have enough voucher to buy the good"<<endl<<endl;
            continue;
        }
        price_payment(price,voucher);           //付錢方法
        if(voucher_count(voucher)!=0)           //剩下的錢為零即不顯示張數
        {
            voucher_show_remain(voucher);
            cout<<endl;
        }
        if(voucher_count(voucher)==0)       //vouchers為零時 顯示You do not remain any voucher，Bye~
        {
            cout<<"You do not remain any voucher"<<endl<<"Bye~"<<endl;
        }
    } //end while
    return 0;
```

大致流程如下:
一開始宣告price陣列存放價格、voucher陣列存放張數、data陣列存放(d/p)和(y/n)

進入while迴圈(條件為price!=-1和voucher!=0):
輸入價格 → 判斷價格是否為-1(為-1離開迴圈) → 輸入用什麼券付和是否為會員 → 計算價格 → 判斷錢夠付嗎(不夠付回到一開始) → 呼叫(函數)付錢方法 →
判斷剩下的錢是否為零:

為零 → 顯示 You do not remain any voucher Bye~
不為零 → 顯示張數

→ 判斷還有沒有錢:

沒錢 → 顯示You don not remain any voucher
有錢 → 繼續迴圈

函數一:輸入價錢(傳入price陣列)

```
void price_set(int price[])                    //     set price
{
    double pricel;
    cout<<"Please enter the good price:";
    do
    {
        cin>>pricel;
        price[0]=pricel;
        if((price[0]<=0&&price[0]!=-1)||pricel>price[0])
        {
            cout<<"Please enter the correct price:";
        }
    }while((price[0]<=0&&price[0]!=-1)||pricel>price[0]);

    return;
}
```

先印出提示字元(Please enter the good price:)
使用do-while迴圈
如果價錢沒在範圍內，則重新輸入
判斷條件為:price <=0並且price!=-1或price為小數

函數二:設置資料(傳入data陣列)

```
void data_set(string data[])          //set data
{
    do
    {
        cout<<"Please enter the type of vouchers (d/p):";
        cin>>data[0];
        if(data[0]!="d"&&data[0]!="p")
            cout<<"Please enter the correct type of vouchers"<<endl;
        if(data[0]=="d")
        {
            do
            {
                cout<<"Are you a member of NCU Bank (y/n):";
                cin>>data[1];
                if(data[1]!="y"&&data[1]!="n")
                    cout<<"Please input the correct data"<<endl;
            }while(data[1]!="y"&&data[1]!="n");

        }
        else
        {
            data[1]="n";
        }
    }while(data[0]!="d"&&data[0]!="p");

    return;
```

data[0]存放(d/p) 、data[1]存放(y/n)
使用巢狀迴圈(包含兩個do while)
大的迴圈判斷條件為data[0]是否為d/p
如果data[0]為d，進入小迴圈，小迴圈判斷條件為data[1]是否為y/n

函數三:價格計算(傳入 price陣列和data陣列)

```
void price_count(int price[],string data[])    //count price
{
    if(data[0]=="p")
        price[0]=price[0]*0.9;
    else if(data[1]=="y")
            price[0]=price[0]*0.8*0.9;
        else
            price[0]=price[0]*0.8;

    cout<<"The price you need to pay:"<<price[0]<<endl;
    return;
}
```

如果data[0]為p則pricet乘0.9(九折)
不為p則判斷data[1]為y還是n
為y則price乘0.8在乘0.9(八折再九折)
為n則price乘0.8(打八折)
最後印出你要付的錢為多少

函數四:判斷錢夠付嗎(傳入price陣列和voucher陣列)
傳回值為bool

```
bool price_judge(int price[],int voucher[])   //judge price
{
    if(price[0]>voucher_count(voucher))
        return false;
    else
        return true;
}
```

判斷voucher是否足夠支付price
(這函數裡有呼叫另一個函數voucher_count，它的功用主要是計算voucher有多少錢)
不夠付 傳回false
夠付　傳回true

函數五:付錢方法
(void price_payment(int price[],int voucher[]))

這個是最主要，也是我認為最難寫的函數， 在 hardness of this assignment會做介紹 。

函數六:顯示剩餘張數(傳入voucher陣列)

```
void voucher_show_remain(int voucher[])     //show vouchers
{
    cout<<"The remaining vouchers are as follows:"<<endl;
    if(voucher[4]!=0)
        cout<<"     "<<voucher[4]<<" $ 100 voucher"<<endl;
    if(voucher[3]!=0)
        cout<<"     "<<voucher[3]<<" $  50 voucher"<<endl;
    if(voucher[2]!=0)
        cout<<"     "<<voucher[2]<<" $  10 voucher"<<endl;
    if(voucher[1]!=0)
        cout<<"     "<<voucher[1]<<" $   5 voucher"<<endl;
    if(voucher[0]!=0)
        cout<<"     "<<voucher[0]<<" $   1 voucher"<<endl;
    return;

}
```

Voucher[0]代表1的張數
Voucher[1]代表5的張數
Voucher[2]代表10的張數
Voucher[3]代表50的張數
Voucher[4]代表100的張數

張數不為0則顯示張數

函數七:顯示使用張數(傳入voucher陣列)

```
void voucher_show_use(int voucher[])
{
    cout<<"The payment is processed as follows:"<<endl;
    if(voucher[4]!=0)
        cout<<"    "<<voucher[4]<<" $ 100 voucher"<<endl;
    if(voucher[3]!=0)
        cout<<"    "<<voucher[3]<<" $  50 voucher"<<endl;
    if(voucher[2]!=0)
        cout<<"    "<<voucher[2]<<" $  10 voucher"<<endl;
    if(voucher[1]!=0)
        cout<<"    "<<voucher[1]<<" $   5 voucher"<<endl;
    if(voucher[0]!=0)
        cout<<"    "<<voucher[0]<<" $   1 voucher"<<endl;
    return;
}
```

Voucher[0]代表1的張數
Voucher[1]代表5的張數
Voucher[2]代表10的張數
Voucher[3]代表50的張數
Voucher[4]代表100的張數

張數不為0則顯示張數

函數八:計算有多少錢(傳入voucher陣列)傳回sum

```
int voucher_count(int voucher[])      //count remain vouchers
{
    int sum;
    sum=voucher[0]*1+voucher[1]*5+voucher[2]*10+voucher[3]*50+voucher[4]*100;
    return sum;
}
```

計算有多少錢並設置給sum，最後傳回sum

The hardness of this assignment and how I overcome it:

函數五:付錢方法
(void price_payment(int price[],int voucher[]))

大概分為兩部分:

第一部分為voucher可以剛好支付price

第二部分為voucher無法剛好支付price

第一部分(可以剛好等於price):

```
void price_payment(int price[],int voucher[])  //the method of payment  using brute force
{   int sum;
    int sum_max=price[0];
    int voucher_quantity,voucher_quantity_min=25;
    int voucher_pay[5]={0,0,0,0,0};
    for(int i=0;i<=voucher[0];i++)                          //1$
    {
        for(int j=0;j<=voucher[1];j++)                      //5$
        {
            for(int k=0;k<=voucher[2];k++)                  //10$
            {
                for(int h=0;h<=voucher[3];h++)               //50$
                {
                    for(int e=0;e<=voucher[4];e++)           //100$
                    {
                        voucher_quantity=i+j+k+h+e;
                        sum=i*1+j*5+k*10+h*50+e*100;
                        if(sum==price[0]&&voucher_quantity_min>=voucher_quantity)
                        {
                            voucher_quantity_min=voucher_quantity;
                            voucher_pay[0]=i;
                            voucher_pay[1]=j;
                            voucher_pay[2]=k;
                            voucher_pay[3]=h;
                            voucher_pay[4]=e;
                        } //end   if

                    }       //end for e loop
                }           //end for h loop
            }               //end for k loop
        }                   //end for j loop
    }                       //end for i loop
    if(voucher_pay[0]!=0||voucher_pay[1]!=0||voucher_pay[2]!=0||voucher_pay[3]!=0||voucher_pay[4]!=0)
    {
        if(price[0]!=0)
            voucher_show_use(voucher_pay);
        voucher[0]=voucher[0]-voucher_pay[0];
        voucher[1]=voucher[1]-voucher_pay[1];
        voucher[2]=voucher[2]-voucher_pay[2];
        voucher[3]=voucher[3]-voucher_pay[3];
        voucher[4]=voucher[4]-voucher_pay[4];

        return;
```

如何找到可以剛好支付的張數:
沒有什麼好的想法，所以我就使用Brute Force(列舉所有可能性)去解決，而我們要找的解為:可以剛好等於price，並且張數為最少的組合。

如果voucher_pay(付的張數)不等於初始值，代表有剛好的法可以支付，並且顯示支付的張數，最後將原本的張數扣掉付的張數之後設值給voucher
函數結束

第二部分(無法剛好等於price):

```
for(int i=0;i<=voucher[0];i++)                    //1$
{
    for(int j=0;j<=voucher[1];j++)                //5$
    {
        for(int k=0;k<=voucher[2];k++)            //10$
        {
            for(int h=0;h<=voucher[3];h++)        //50$
            {
                for(int e=0;e<=voucher[4];e++)    //100$
                {
                    voucher_quantity=i+j+k+h+e;
                    sum=i*1+j*5+k*10+h*50+e*100;
                    if(sum>price[0]&&voucher_quantity_min>=voucher_quantity)
                        if(sum_max<=sum)
                        {
                            sum_max=sum;
                            voucher_quantity_min=voucher_quantity;
                            voucher_pay[0]=i;
                            voucher_pay[1]=j;
                            voucher_pay[2]=k;
                            voucher_pay[3]=h;
                            voucher_pay[4]=e;
                        } //end   if

                }       //end for e loop
            }           //end for h loop
        }               //end for k loop
    }                   //end for j loop
}                       //end for i loop
if(price[0]!=0)
  voucher_show_use(voucher_pay);
  voucher[0]=voucher[0]-voucher_pay[0];
  voucher[1]=voucher[1]-voucher_pay[1];
  voucher[2]=voucher[2]-voucher_pay[2];
  voucher[3]=voucher[3]-voucher_pay[3];
  voucher[4]=voucher[4]-voucher_pay[4];
  return;
```

與第一部分樣使用Brute Force去解決，但不一樣的是我們要找的組合為:
大於price的最少張數，並且金額為最大。
因此在這還多了一個sum_max的判斷

最後將原本的張數扣掉付的張數並設值給voucher
函數結束

Bonus function:主要是增加了一個可以找錢的功能

```
for(int i=0;i<=voucher[0];i++)                          //1$
{
    for(int j=0;j<=voucher[1];j++)                      //5$
    {
        for(int k=0;k<=voucher[2];k++)                  //10$
        {
            for(int h=0;h<=voucher[3];h++)              //50$
            {
                for(int e=0;e<=voucher[4];e++)          //100$
                {
                    voucher_quantity=i+j+k+h+e;         //張數
                    sum=i*1+j*5+k*10+h*50+e*100;
                    if(sum>price[0]&&voucher_quantity_min>=voucher_quantity)
                        if(sum_max<=sum)
                        {
                            sum_max=sum;
                            voucher_quantity_min=voucher_quantity;
                            voucher_pay[0]=i;
                            voucher_pay[1]=j;
                            voucher_pay[2]=k;
                            voucher_pay[3]=h;
                            voucher_pay[4]=e;
                        } //end   if

                }       //end for e loop
            }           //end for h loop
        }               //end for k loop
    }                   //end for j loop
}                       //end for i loop
int difference=sum_max-price[0];
int voucher_change[5]={0,0,0,0,0};
if(price[0]!=0)
{
    voucher_show_use(voucher_pay);
    voucher_change_count(difference,voucher_change);
    voucher_show_change(voucher_change);
}

voucher[0]=voucher[0]-voucher_pay[0]+voucher_change[0];
voucher[1]=voucher[1]-voucher_pay[1]+voucher_change[1];
voucher[2]=voucher[2]-voucher_pay[2]+voucher_change[2];
voucher[3]=voucher[3]-voucher_pay[3]+voucher_change[3];
voucher[4]=voucher[4]-voucher_pay[4]+voucher_change[4];

return;
```

在函數五:付錢方法的第二部分做了一些改動
(void price_payment(int price[],int voucher[]))

```
int difference=sum_max-price[0];
int voucher_change[5]={0,0,0,0,0};
if(price[0]!=0)
{
   voucher_show_use(voucher_pay);
   voucher_change_count(difference,voucher_change);
   voucher_show_change(voucher_change);
}

voucher[0]=voucher[0]-voucher_pay[0]+voucher_change[0];
voucher[1]=voucher[1]-voucher_pay[1]+voucher_change[1];
voucher[2]=voucher[2]-voucher_pay[2]+voucher_change[2];
voucher[3]=voucher[3]-voucher_pay[3]+voucher_change[3];
voucher[4]=voucher[4]-voucher_pay[4]+voucher_change[4];

return;
```

要找的錢(difference)是付的錢(sum_max)扣掉price

voucher_change陣列:儲存找的錢的張數

呼叫vooucher_change_count函數(將差額用券表示)

最後將原本的張數扣掉付的張數再加上找的張數並設值給voucher

函數結束

函數void voucher_change_count(int difference,int voucher[]) 內容

```
void voucher_change_count(int difference,int voucher[])
{
    int fifty,ten,five,one;
    voucher[4]=difference/100;
    fifty=difference%100;
    voucher[3]=fifty/50;
    ten=fifty%50;
    voucher[2]=ten/10;
    five=ten%10;
    voucher[1]=five/5;
    one=five%5;
    voucher[0]=one;
    return;
}
```

將差額(difference)與voucher_change陣列傳入函數(body):
將difference換算成券(voucher)，從大面額的開始換

Any suggestions about this programming assignment:

No